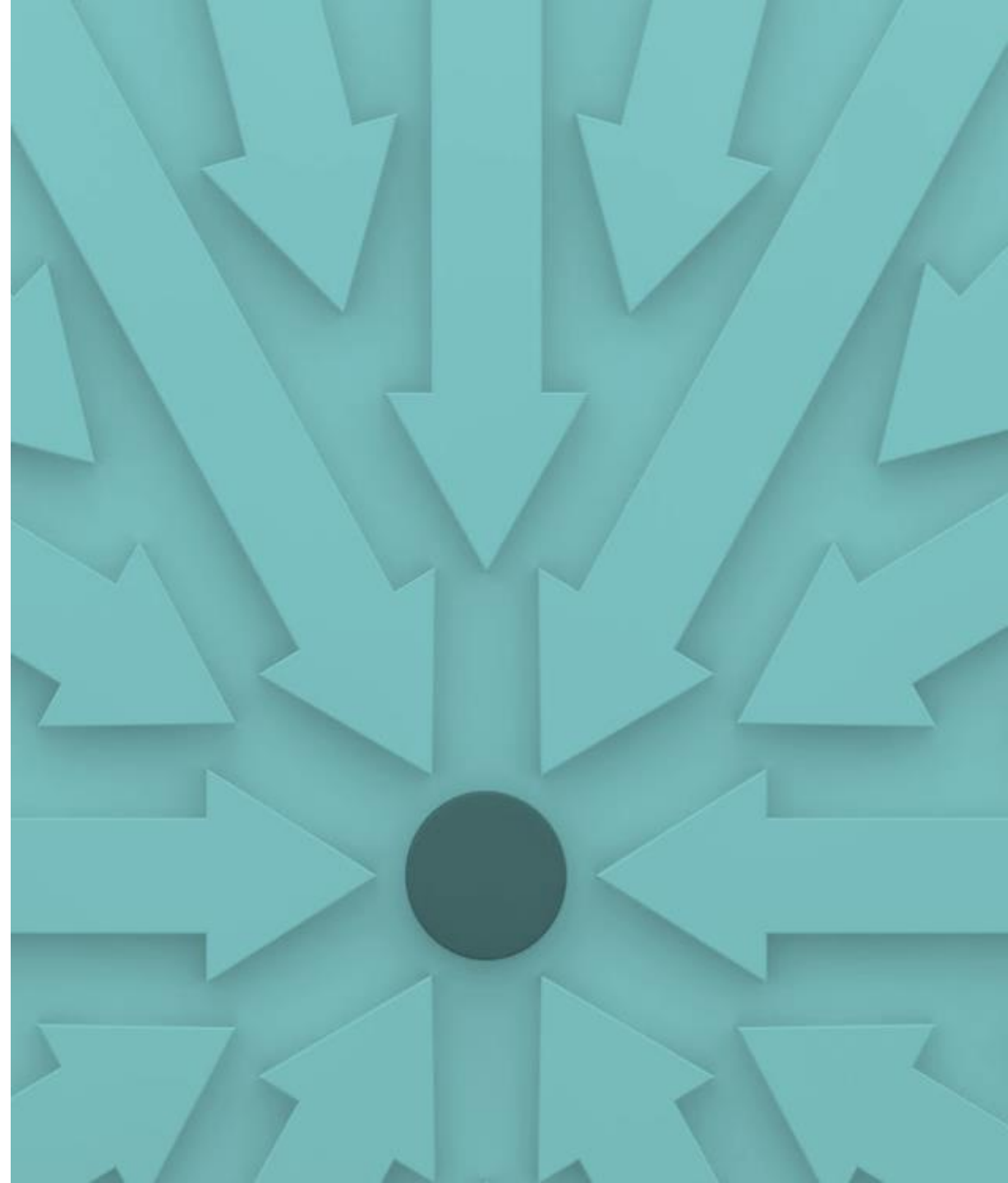


A butterfly with red and black wings is perched on a cluster of small white flowers. The background is a soft, out-of-focus grey.

AI-Assisted Development

Dr. Kornelia Papp

Build with AI to
accelerate
discovery, not just
to deliver code.



Agenda

01

Session goals

02

The AI-assisted landscape

03

Development with Claude Code

04

Experience AI-assisted development

Session Goals

In this session, we explore how AI-assisted development can accelerate product discovery, prototyping, coding, documentation, and retrieval-based AI systems while keeping human judgement in control.

What are AI coding agents?

Understand how AI coding agents support software work by writing code, fixing bugs, explaining codebases, generating documentation, and helping developers move faster.

How can Claude Code support real development workflows?

Learn how Claude Code works across the terminal, IDE, browser, desktop, and mobile app, and how it can help with multi-file edits, debugging, testing, version control, MCP integrations, and project automation.

How can AI support product discovery and data preparation?

Use AI-assisted development to collect structured data, evaluate data quality, identify risks, and prepare datasets for AI use cases.

How can retrieval systems improve AI products?

Build intuition for naïve RAG, FAISS, and retrieval quality, and learn to diagnose whether failures come from the model, the prompt, the retrieved context, or the underlying data.

Agenda

01

Session goals

02

The AI-assisted Landscape

03

Development with Claude Code

04

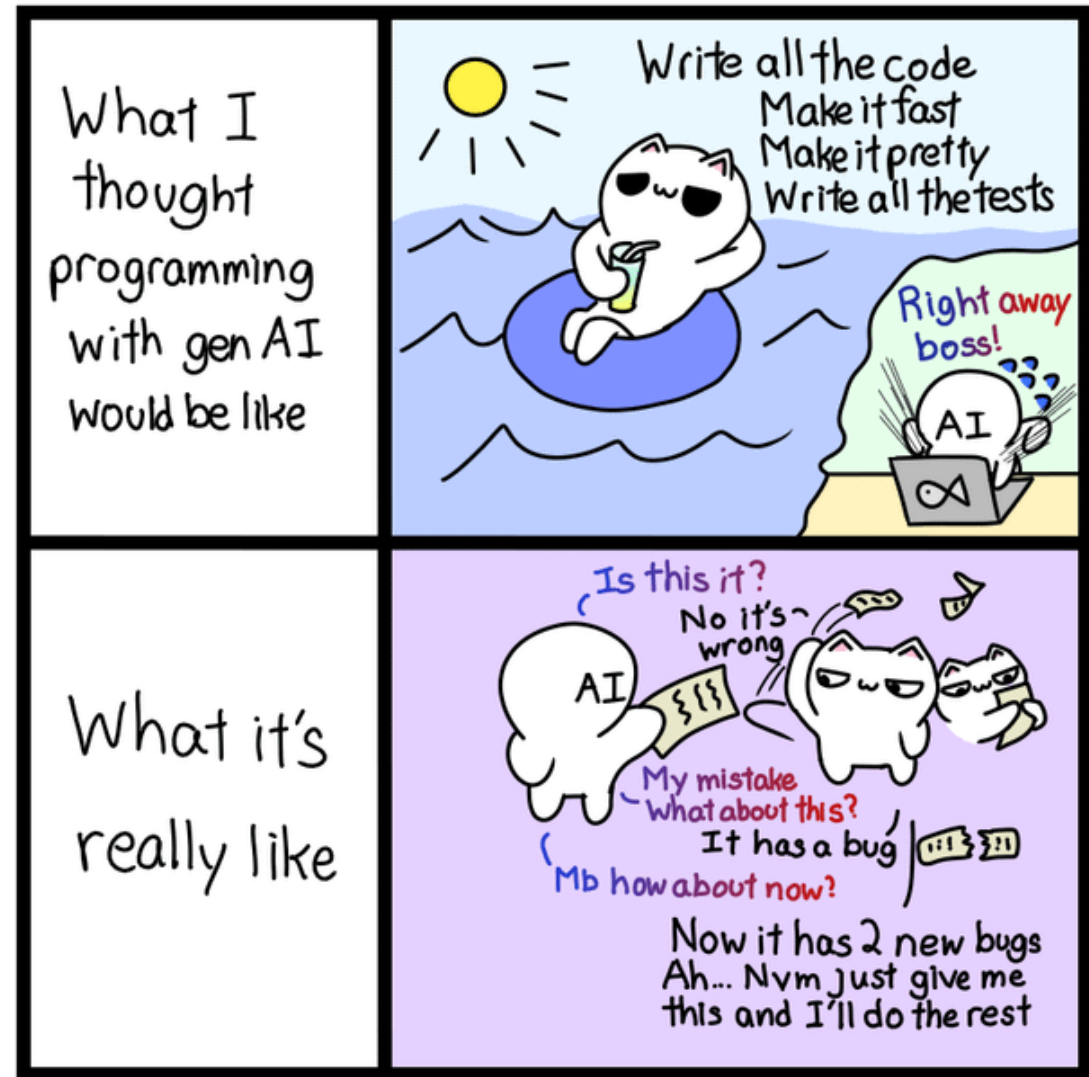
Experience AI-assisted development

AI Coding Agents

AI coding agents are smart programs that understand natural language and programming languages.

They can:

- Write or complete your code automatically.
- Fix bugs or suggest improvements.
- Explain the code line by line.
- Generate documentation or comments.
- Help you learn new programming languages.



AI Coding Agents

When I look at my conversations with the agents, it's very clear to me that this is moon language for the vast majority of human beings.

There are a whole bunch of reasons I'm not scared that my career as a software engineer is over now that computers can write their own code, partly because these things are amplifiers of existing experience. If you know what you're doing, you can run so much faster with them. [...]

(Simon Willison, May 2026)

AI-assisted Landscape

Discovery-oriented tools: **vibe-first** development

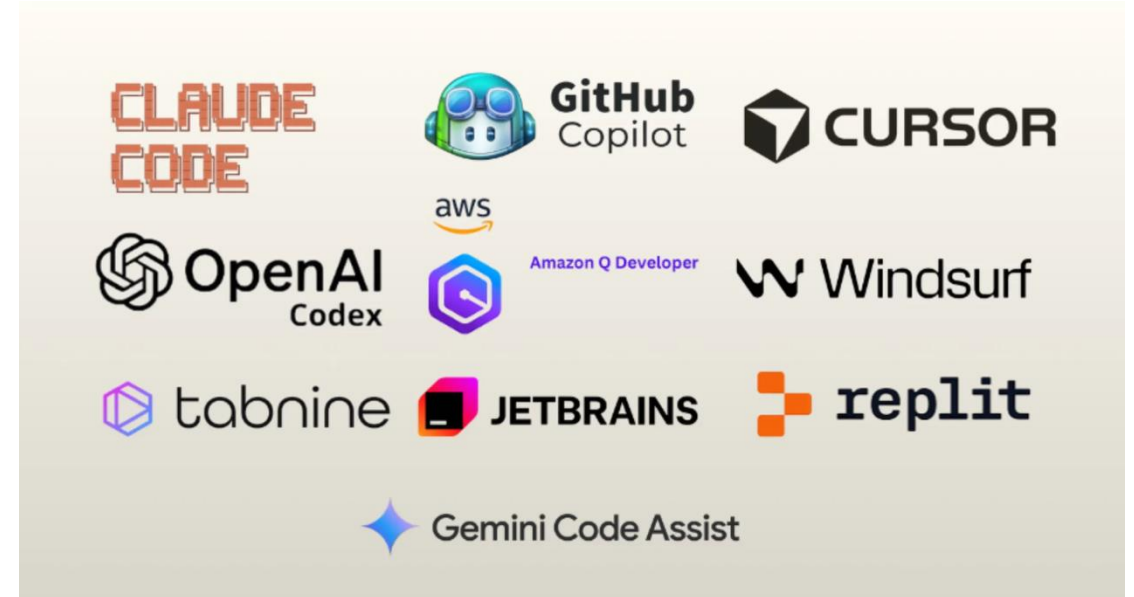
Tools such as Lovable or Replit Agent are well suited to early-stage exploration, where prototypes are developed while the underlying opportunity, user need, and solution space are still being clarified.

Control-oriented tools: **engineer-first** development

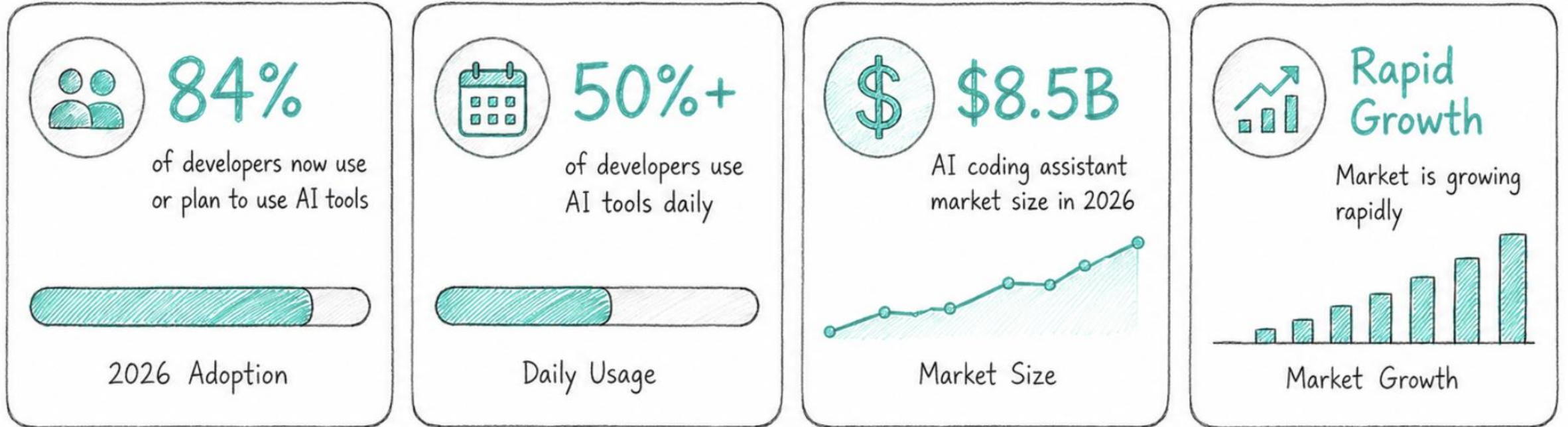
Tools such as Claude Code are designed for software engineering workflows that require greater control over the codebase, architecture, implementation details, and development process.

Shift in workflow and discovery

In AI-assisted development, discovery extends beyond identifying user pain points. It also involves observing where AI can reliably support or automate specific parts of a manual process, such as inspecting information, making bounded decisions, or summarizing complex inputs.



AI-assisted Landscape



AI-assisted Landscape



Best Frontend

v0

98% score



Best Backend

Cursor

95% score



Easiest to Edit

Cursor

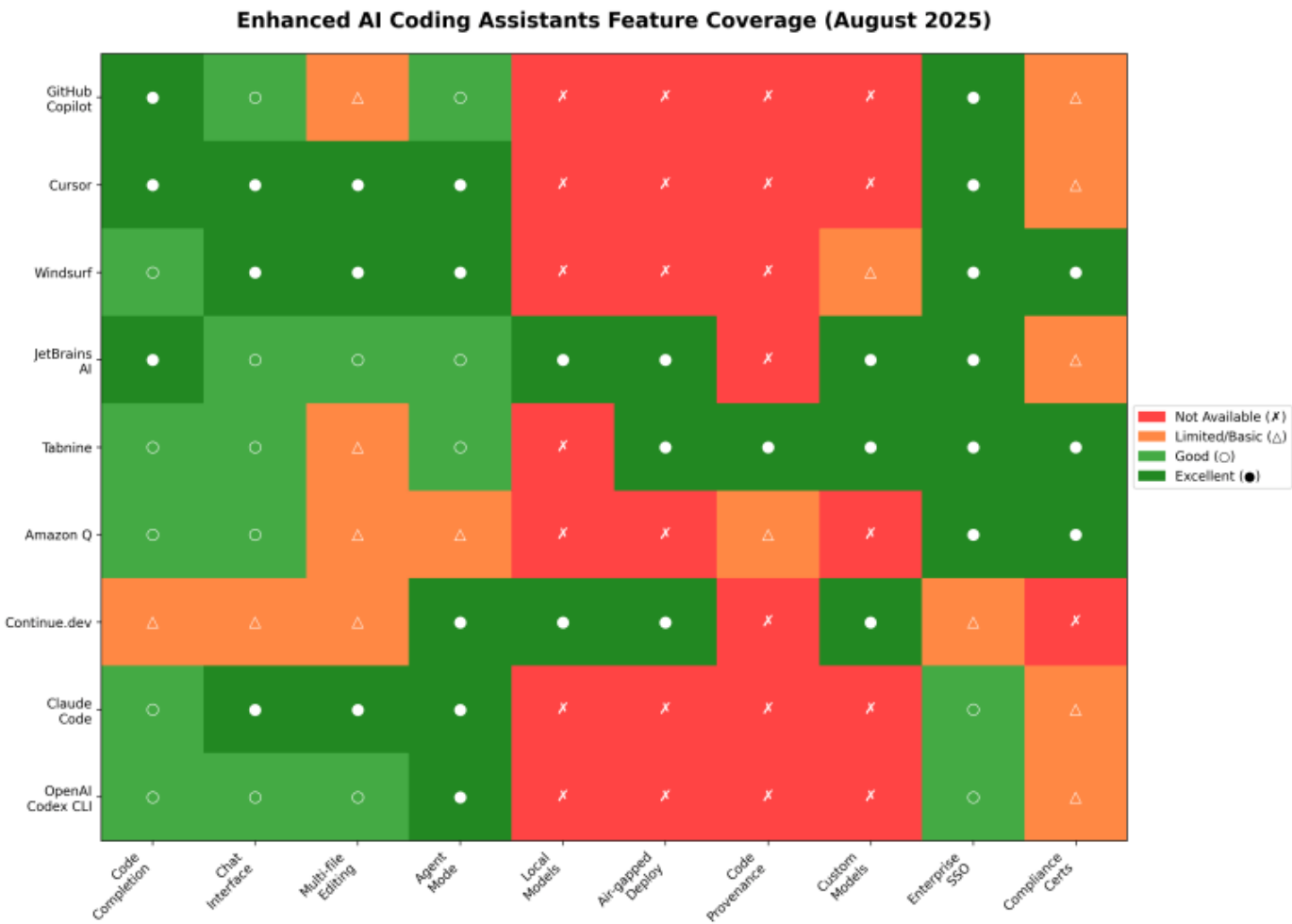
90% score



Best Value

v0

95% score



Source: <https://lawrence.eti.br/2025/08/10/the-new-era-of-ai-coding-assistants-comparing-models-and-tools-in-2025/>

AI Agent Profiling

Tool	Profile	Best For...	Control Level
Lovable	Outcome-First	Non-technical builders needing a fast, full-stack MVP (UI, Auth, DB) with zero setup.	Low: You focus on the outcome; it handles the code. High "Time to Impact."
Replit Agent	Platform-First	Builders who want a cloud-hosted sandbox to experiment, build, and deploy in one tab.	Medium: Great for "Research/Prototypes" where hosting is as important as coding.
Claude Code	Logic-First	Professionals who want a command-line agent to perform precise, multi-file edits across an entire codebase.	High: The gold standard for "Structural Advantage." You are the editor; the AI is the super-powered assistant.
Cursor	Context-First	People who want to co-pilot a codebase with deep AI awareness of every file, function, and doc.	

Agenda

01

Session goals

02

The AI-assisted landscape

03

Development with Claude Code

04

Experience AI-assisted development

Claude Code

Claude Code is an AI-powered coding assistant that helps you build features, fix bugs, and automate development tasks. It understands your entire codebase and can work across multiple files and tools to get things done. It is available in your terminal, IDE, desktop app, and browser.

- Automate deferred or repetitive development tasks.
- Support feature implementation and bug resolution.
- Assist with version-control workflows, including commits and pull requests.
- Integrate external tools and services through MCP.
- Enable customization through instructions, skills, and hooks.
- Support agent-based workflows, including agent teams and custom agents.
- Provide command-line automation through piping, scripting, and CLI integration.
- Schedule recurring development or maintenance tasks.
- Enable work across different environments and locations.



How Claude Code Works

1. The LLM understands the task

You describe what you want in natural language.

Example: “Add a search bar to my book recommender app.”

2. Tools give it access to your project

Claude Code can read your files, understand the project structure, and find the relevant code.

Example: It opens App.tsx, checks the components folder, and finds where the book list is rendered.

3. It edits code and runs commands

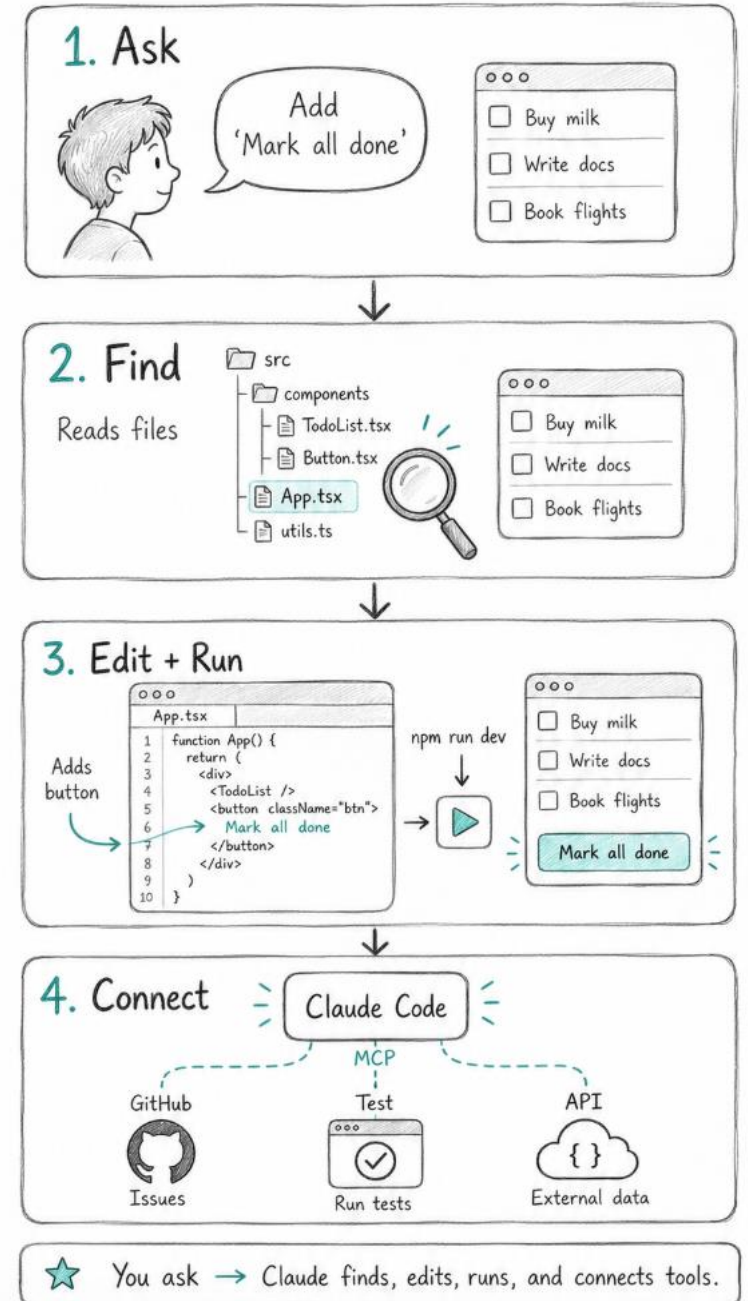
It can modify files, create new components, install packages, and run tests or the app.

Example: It creates SearchBar.tsx, updates App.tsx, and runs npm run dev.

4. MCP connects external tools

Through MCP servers, Claude Code can work with tools outside the local project.

Example: It can open a GitHub issue, test the app with Playwright, or query a database/API.



Claude Code Setup

Claude Code is no longer limited to the terminal!

Local CLI

For terminal-first coding directly inside a local project.

VS Code / IDE integration

For editor-integrated workflows with code context, diffs, and project navigation.

Desktop app

For app-based use through Claude Desktop.

Web — claude.ai/code

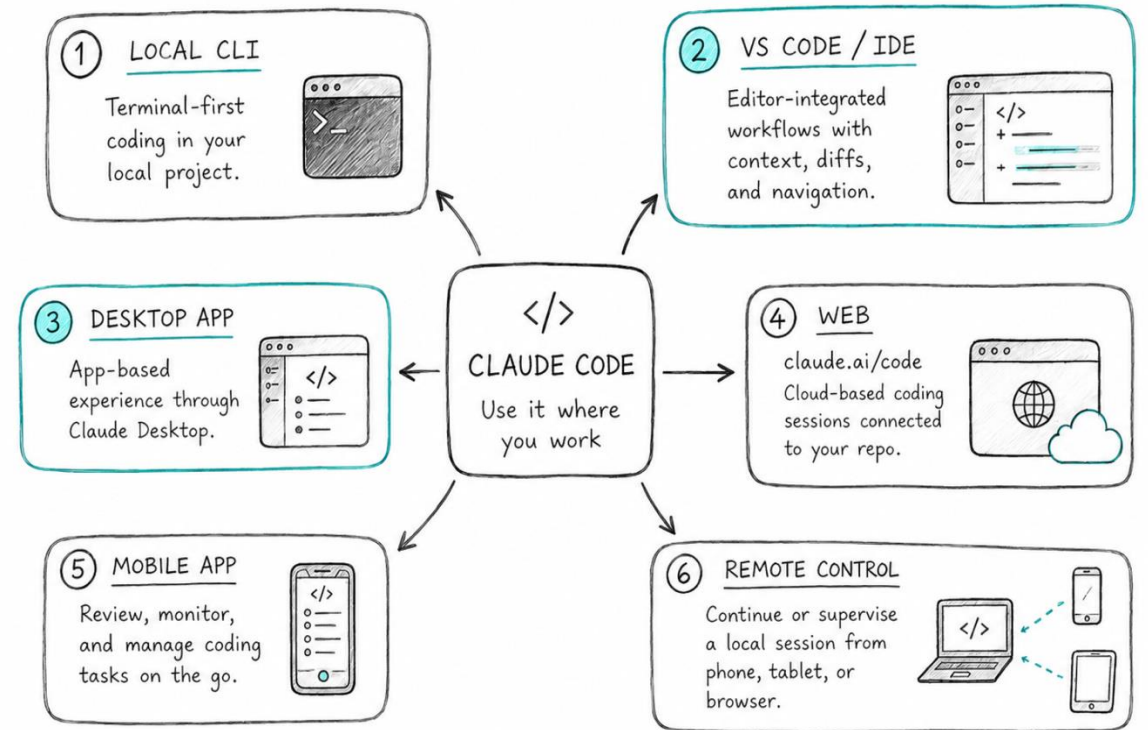
For cloud-based coding sessions connected to repositories.

Mobile app

For reviewing, monitoring, and managing coding tasks on the go.

Remote Control

For continuing or supervising a local Claude Code session from a phone, tablet, or browser.



Choose setup based on control needs: local for codebase control, web/mobile for supervision.

Claude Code Models & Settings

Default / Sonnet

Best for everyday development work: code edits, explanations, debugging, refactoring, and normal agentic coding tasks. Good balance between capability, speed, and usage.

Opus

Most capable option for complex work: difficult debugging, architectural decisions, multi-file refactoring, ambiguous tasks, and high-stakes reasoning. Higher usage than Sonnet.

Haiku

Fastest option for quick answers: simple explanations, lightweight edits, short lookups, and low-complexity tasks. Best when speed matters more than depth.

Effort

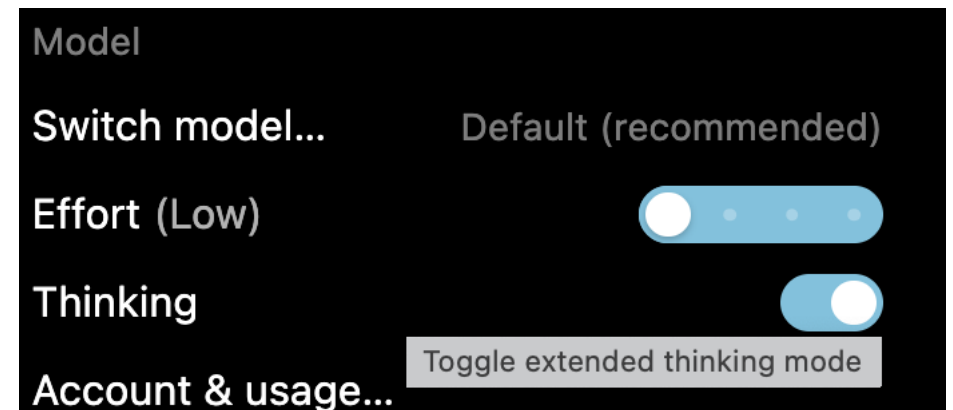
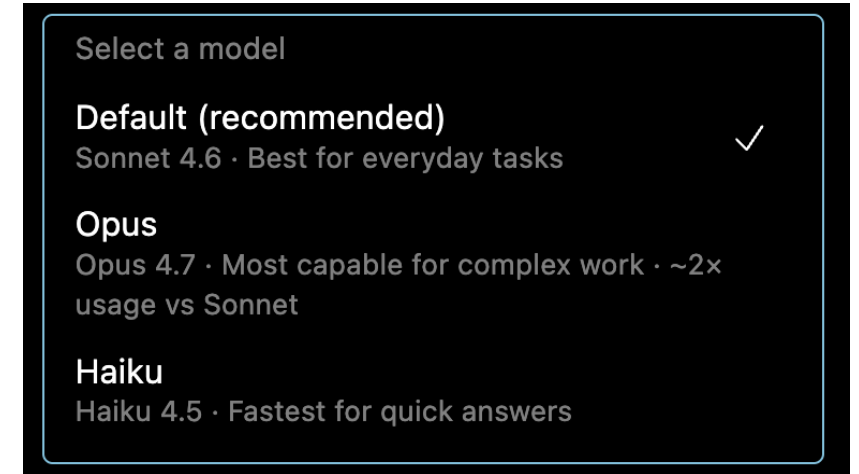
Controls how much work the model puts into solving the task.

Thinking / extended thinking

Enables deeper reasoning for harder tasks. Anthropic's current guidance recommends adaptive thinking with an effort parameter for newer Claude models, especially for agentic behaviour, multi-step tool use, complex coding tasks, and long-horizon workflows.

Account & usage

Shows plan and usage information, helping you monitor consumption when using more capable models or higher-effort settings.



Customization

Customize

Output styles



Agents



Hooks



Memory



Permissions



MCP servers

Manage plugins

Open Claude in Terminal



Output styles: change how Claude responds, for example concise, explanatory, or code-focused.

Agents: configure specialized AI agents for different tasks.

Hooks: set up commands that run automatically at certain moments.

Memory: manage saved instructions Claude should remember.

Permissions: control what Claude is allowed to do, such as reading files, editing files, or running commands.

MCP servers: connect external tools through Model Context Protocol, such as GitHub, Playwright, databases, or APIs.

Manage plugins: install or manage Claude Code extensions.

Context Management

Context management is very important when working with AI coding tools. The clearer and more specific your prompt is, the better the output will be. You should always point Claude to the relevant files. This helps it focus on the right part of the project instead of wasting tokens searching through unrelated code.

Claude Code also provides [commands and shortcuts to manage context](#) during a coding session:

/context

Shows how the context window is being used, including heavy context sources and optimization suggestions. Use it when the session feels long, slow, or confused.

/compact [instructions]

Summarizes the conversation so far to free context while preserving important state. Best used at milestones: after finishing a feature, before starting a new large task, or when context is getting full. You can guide it:

`/compact keep architecture decisions, TODOs, test commands, and open bugs.`

/clear [name]

Starts a new conversation with empty context while keeping project memory. Use it when switching to an unrelated task. The previous conversation remains available through `/resume`.

Context Management (Favourites)

/memory

Lets you edit CLAUDE.md memory files, manage auto-memory, and view what Claude has remembered. Use it for persistent project context: build commands, coding conventions, architecture notes, and repeated corrections.

/init

Creates or improves a project CLAUDE.md. This is important because CLAUDE.md is loaded into context at the start of sessions and gives Claude durable project knowledge.

/resume [session]

Returns to a previous conversation by ID or name. Useful when you intentionally separated tasks into different sessions instead of overloading one context.

/branch [name]

Forks the current conversation at this point. Useful when you want to explore an alternative solution without polluting the main thread.

/btw <question>

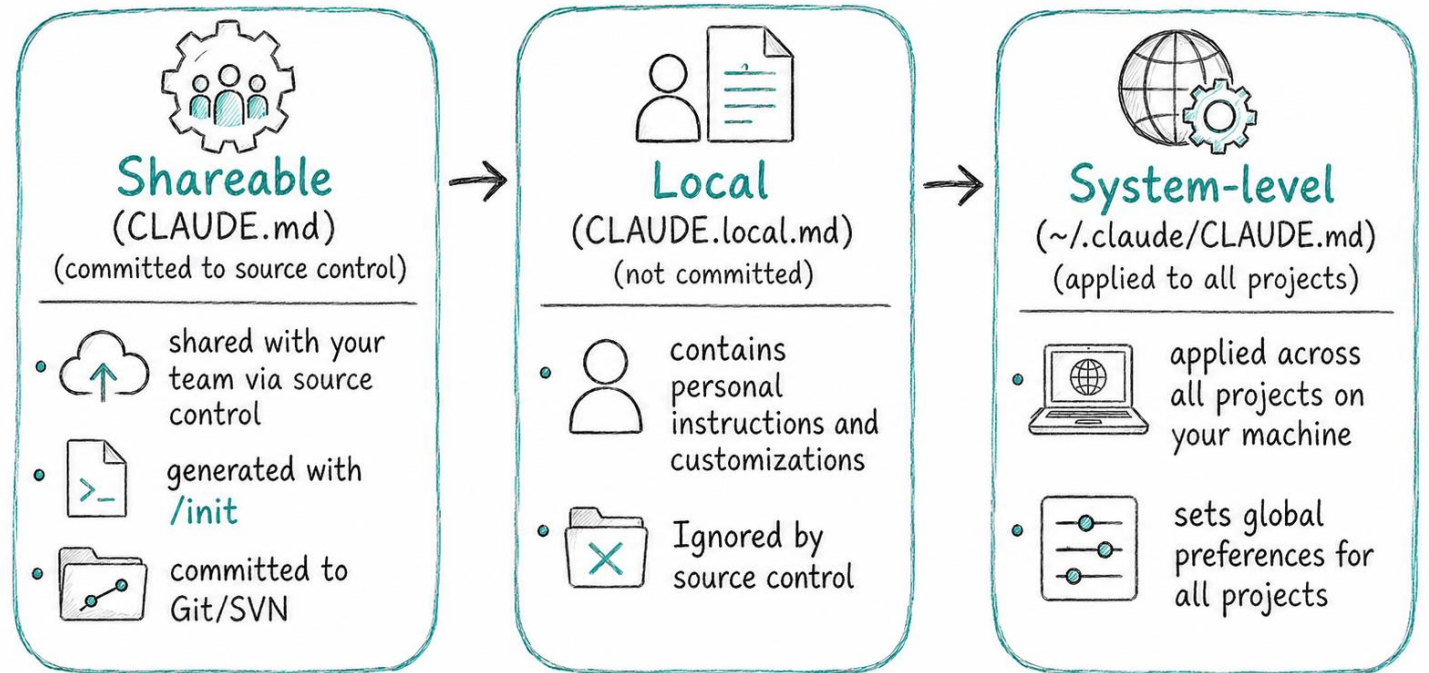
Asks a side question without adding it to the main conversation history. Very useful for quick clarifications that should not bloat or distract the working context.

Context Management

CLAUDE.md is a project-specific memory and instruction file used by Claude Code.

Provides Claude with important information about your codebase, such as:

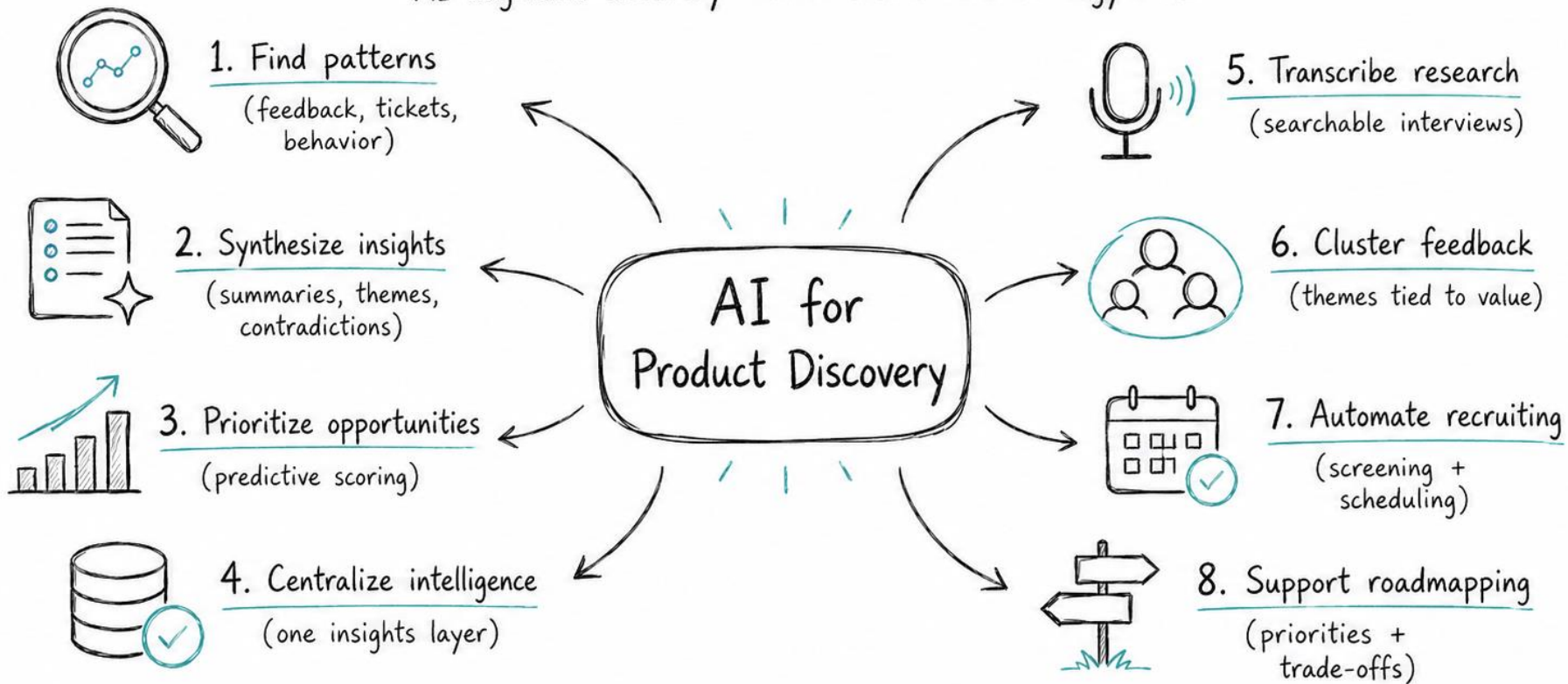
- project structure
- coding conventions
- build and test commands
- architecture notes
- preferred workflows
- common mistakes to avoid
- ...



Before building, use AI to structure what should be built and what evidence is needed

How AI Empowers Product Discovery

AI augments discovery — humans still make strategy calls.



☆ More signal, faster learning, better decisions — but strategy remains human-led. ☆

Agenda

01

Session goals

02

The AI-assisted landscape

03

Development with Claude Code

04

Experience AI-assisted development



Explore & Discuss

Exercise: **Collect Data for an AI Use Case**

Use Claude Code (or similar tools) to write a Python script to pull data from the web.

Requirements:

- Fetch data from a public online source
- Save structured data as CSV

Inspiration

KEEP IT SIMPLE

Think about ->

What data is needed?

What documents, user inputs, examples, feedback, logs, or labels are available?

Where would the data come from?

Internal documents, public sources, surveys, interviews, support tickets, product usage, APIs, or synthetic examples?

Is the data good enough?

Is it complete, accurate, recent, representative, and structured enough?

What should not be collected?

Personal data, sensitive information, unnecessary user details, copyrighted content, or anything outside the legitimate purpose.

How would quality be checked?

Manual review, sampling, validation rules, duplicate detection, human feedback, or benchmark examples.

Music / Poetry Data

Music data

README: <https://github.com/droplinger/music-data/blob/main/README.md>

Music-data:

<https://github.com/droplinger/music-data/blob/main/README.md>

https://github.com/droplinger/music-data/blob/main/get_playlist_data.py

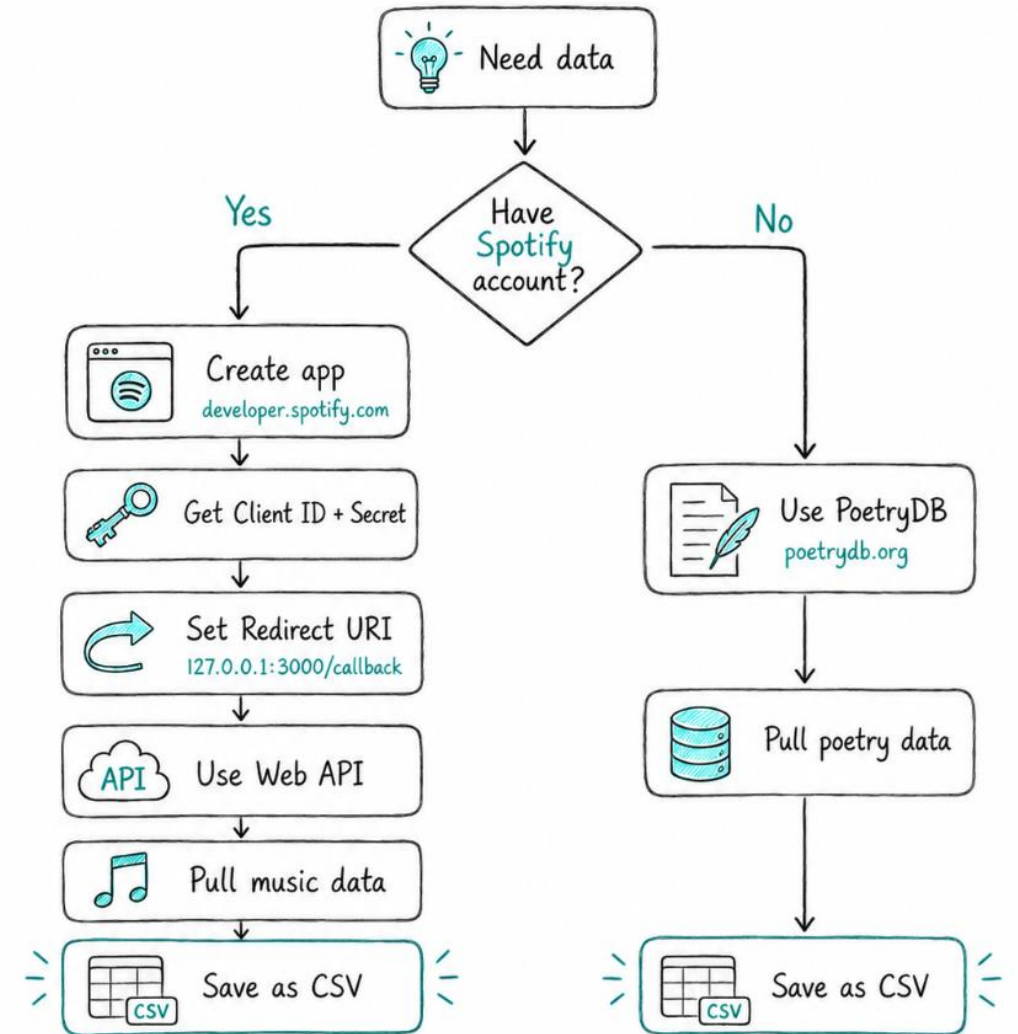
Poetry data:

README: <https://github.com/droplinger/poetry-data/blob/main/README.md>

Poetry-data: https://github.com/droplinger/poetry-data/blob/main/get_poetry_data.py

Choose a Data Source

Music with Spotify or poetry with PoetryDB



Get Your Music Data

Create a Spotify Developer app: developer.spotify.com

Go to the Spotify Developer Dashboard, create an app, and note the **Client ID** and **Client Secret**. Spotify uses these to identify your app.

Set a Redirect URI

Example for local development: <http://127.0.0.1:3000/callback>

This must match exactly in your app settings and your code.

Set APIs used: Web API

Pick a **public** playlist:

Open the playlist in Spotify, then: **Desktop app / web player**

Open the playlist. -> Click ... -> Choose **Share** -> Choose **Copy link to playlist**

You'll get something like: <https://open.spotify.com/playlist/37i9dQZF1DXcBWIGoYBM5M?si=abc123>

The playlist ID is the part after /playlist/ and before ?: 37i9dQZF1DXcBWIGoYBM5M

So this API URL would be: <https://api.spotify.com/v1/playlists/37i9dQZF1DXcBWIGoYBM5M/tracks>

Spotify Web API

Note: You need a Spotify Premium account to use the Web API.

1. Log in to the [dashboard](#) using your Spotify account.

2. [Create an app](#) and select "Web API" for the question asking which APIs are you planning to use. Once you have created your app, you will have access to the app credentials. These will be required for API [authorization](#) to obtain an [access token](#).

3. Use the [access token](#) in your [API requests](#).

Playlist documentation: <https://developer.spotify.com/documentation/web-api/reference/get-playlist>

- Limit is 50 items max.



Public Playlist

Best of SingSong

+ gruggugur • 100 songs, about 6 hr 30 min

Singer-songwriter <https://open.spotify.com/playlist/1YVBCp4xDQdl4HGTtml8mb?si=oSt1z3eJShmOxQ14R4wNcw>



Public Playlist

Best of Rap

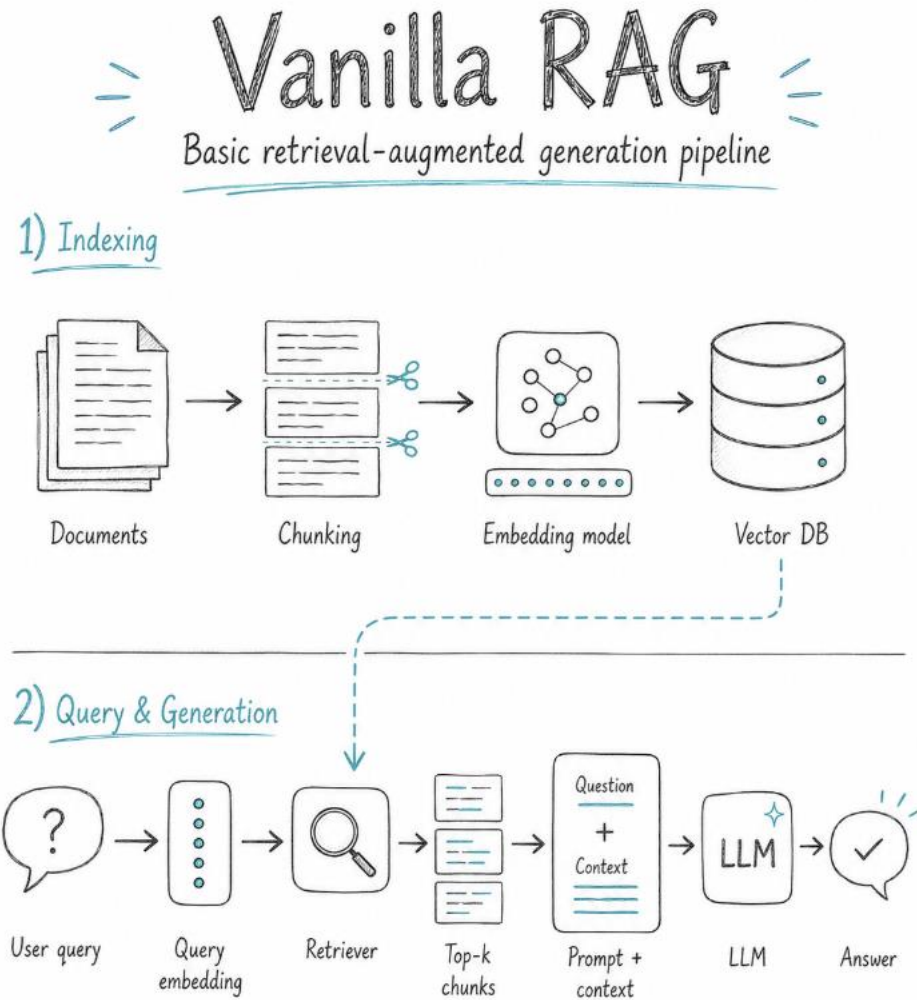
+ gruggugur • 100 songs, about 7 hr

Rap: <https://open.spotify.com/playlist/2bucOWIEc5FMfMLPoqUPue?si=kkUalPaWT7mKeJ1NhkArmQ>

Naïve RAG

Naïve RAG is a basic architecture where documents are split into chunks, converted into embeddings, stored in a vector database, and retrieved based on similarity to the user's question.

The **most relevant chunks** are then passed to the language model, which uses them to generate an answer.



RAG Versions

1. Hybrid RAG

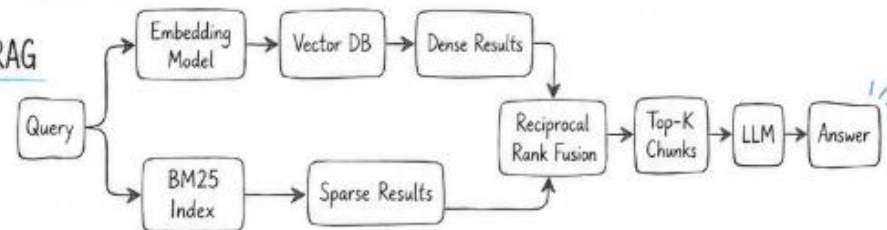
Hybrid RAG combines dense semantic retrieval with sparse keyword-based retrieval. Dense retrieval captures conceptual similarity through embeddings, while sparse retrieval, such as BM25, preserves exact lexical matching. The results are then fused, often by reciprocal rank fusion, to improve recall and robustness before passing the top-ranked chunks to the LLM.

2. GraphRAG

GraphRAG structures information as a knowledge graph of entities and relationships. Instead of retrieving isolated text chunks, it retrieves subgraphs or community summaries that represent relational context. This is especially useful when the answer depends on connections between people, concepts, locations, events, or causal chains.

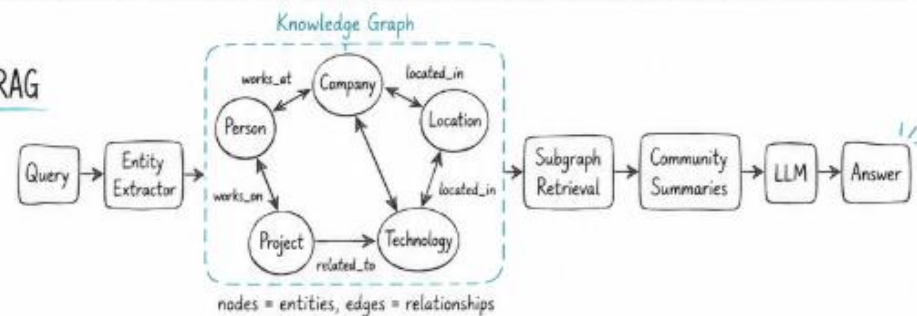
01 HYBRID RAG

Dense vectors meet sparse keywords.



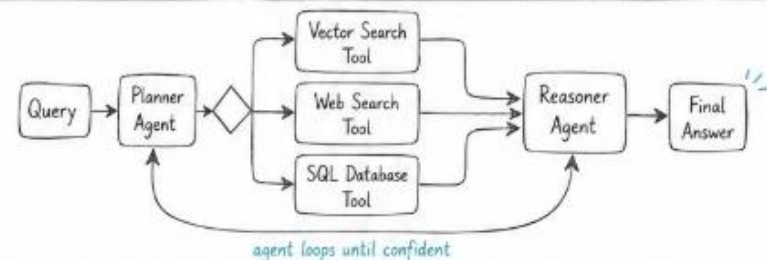
02 GRAPH RAG

Answers live in the relationships.



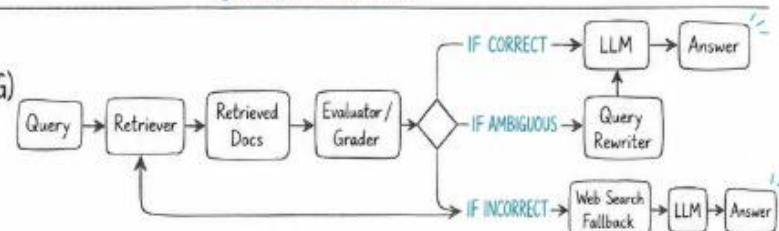
03 AGENTIC RAG

Retrieval becomes a plan, not a step.



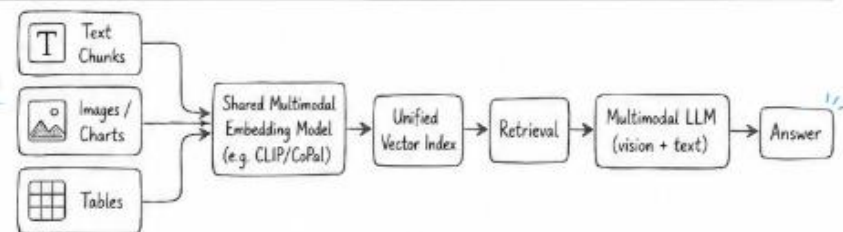
04 CORRECTIVE RAG (CRAG)

Grade the retrieval before you trust it.



05 MULTIMODAL RAG

One index across text, images, and tables.



RAG Versions

3. Agentic RAG

Agentic RAG treats retrieval as a dynamic planning process rather than a single retrieval step. A planner or reasoning agent decides which tools to use, such as vector search, web search, or databases, and may iterate until sufficient evidence is collected. This architecture is suitable for complex, multi-step questions requiring judgment and adaptive information gathering.

4. Corrective RAG / CRAG

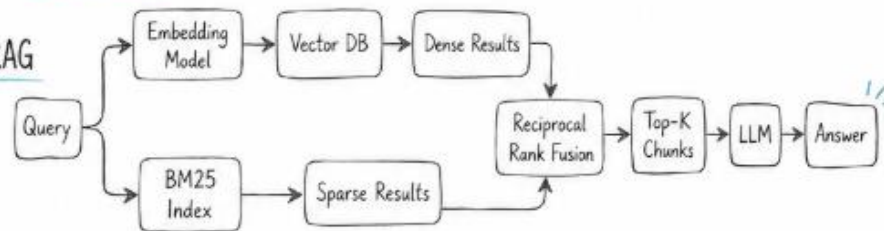
Corrective RAG introduces an evaluation stage after retrieval. Retrieved documents are graded for relevance or reliability before being used by the LLM. If the retrieval is weak, ambiguous, or incorrect, the system may rewrite the query or fall back to external search. Its main goal is to reduce hallucination caused by poor retrieval.

5. Multimodal RAG

Multimodal RAG extends retrieval beyond text to include images, charts, tables, and other data modalities. A shared multimodal embedding model maps different content types into a common representation space. The system can then retrieve relevant evidence across modalities and provide it to a multimodal LLM for answer generation.

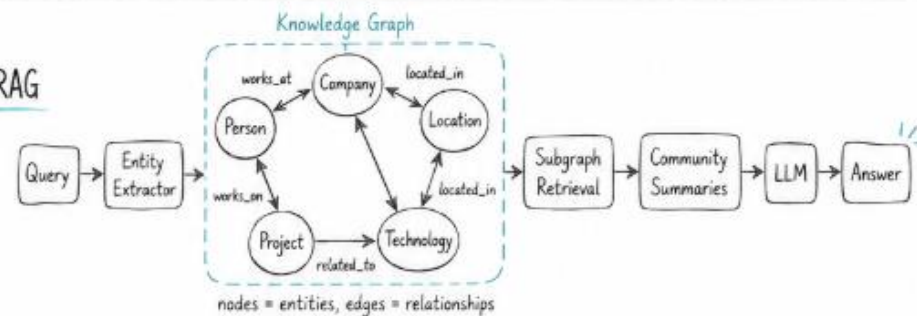
01 HYBRID RAG

Dense vectors meet sparse keywords.



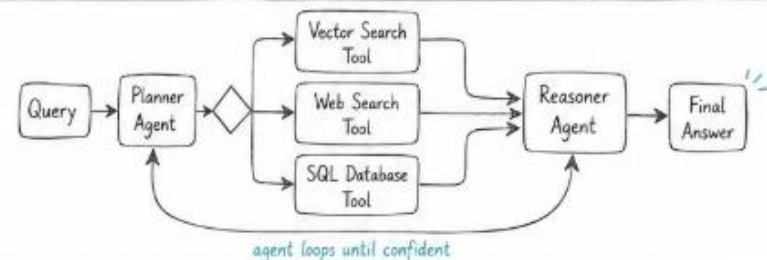
02 GRAPH RAG

Answers live in the relationships.



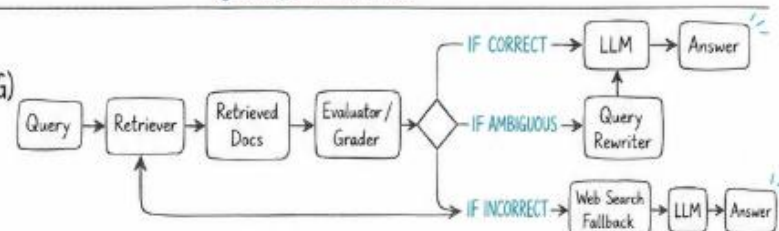
03 AGENTIC RAG

Retrieval becomes a plan, not a step.



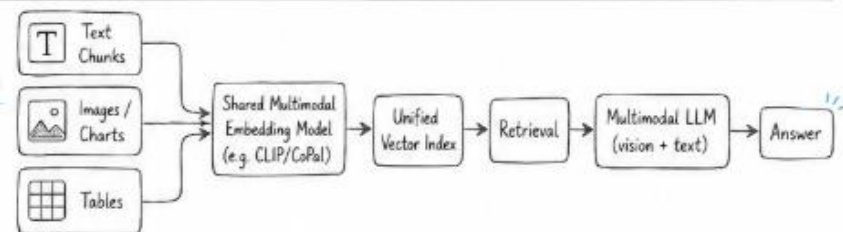
04 CORRECTIVE RAG (CRAG)

Grade the retrieval before you trust it.



05 MULTIMODAL RAG

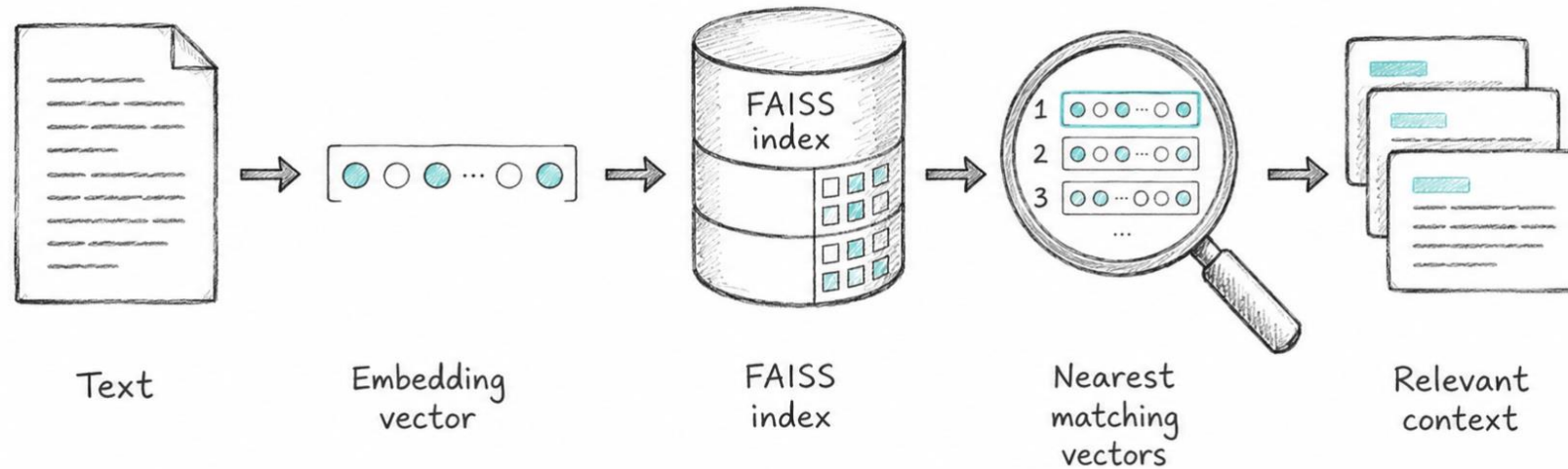
One index across text, images, and tables.



FAISS

FAISS = Facebook AI Similarity Search

FAISS is an open-source library for fast similarity search and clustering of dense vectors. It is commonly used in RAG systems to retrieve the most relevant text chunks before an LLM answers.



Traditional search is good at exact keyword matching. FAISS supports semantic search, where items can be retrieved because they are similar in meaning or representation, not because they share the same words.

FAISS

FAISS stores embedding vectors in an index and retrieves the nearest vectors to a query vector.

Techniques behind the speed

- **K-means clustering** narrows the search space by grouping similar vectors.
- **Product quantization** compresses vectors to reduce memory use.
- **GPU support** can accelerate search for large-scale workloads.

FAISS is valuable because it balances **speed, scale, memory use, and accuracy**.

Key strengths

- Handles millions to billions of vectors
- Supports CPU and GPU execution
- Works with different similarity measures
- Can trade off accuracy against speed
- Useful for real-time retrieval tasks

FAISS

FAISS in a RAG System

In RAG, FAISS helps the LLM find relevant information before generating an answer.

RAG pipeline

Question → embedding → FAISS search → relevant chunks → LLM answer

Example:

Question:

“What is Switzerland’s largest city?”

Retrieved chunk:

“Zürich is Switzerland’s largest city.”

Generated answer:

“Zürich is Switzerland’s largest city.”

FAISS does not generate answers. It retrieves relevant context efficiently, so the LLM can answer with better grounding.



Exercise

Exercise: **Build a Retrieval System**

Design a small RAG system that answers questions from a selected document collection, for example lecture notes, company policies, product documentation, scientific papers, or FAQs.

Requirements:

- Backend is available
- Frontend is available (optional)

Think about:

Is the problem with the answer caused by the model, the prompt, the retrieved context, or the underlying data?

Help: <https://github.com/droplinger/rag>

Inspiration

KEEP IT SIMPLE

Think about ->

What should go into the knowledge base?

Which documents are relevant, trustworthy, up to date, and useful for answering questions?

How should documents be split?

Should chunks be short paragraphs, sections, pages, Q&A pairs, or topic-based units?

What should be retrieved?

How many chunks should the system retrieve? Should it retrieve exact matches, semantically similar passages, or both?

How should the model answer?

Should it summarize, cite sources, quote evidence, admit uncertainty, or say “not found” when the answer is missing?

How would failure be detected?

Wrong answer, missing citation, irrelevant retrieval, outdated source, hallucinated claim, or answer not grounded in the retrieved text.

How would the system be evaluated?

Use test questions, expected answers, source checking, human review, and examples where the correct answer is not in the documents.



Try (simple RAG)

Add more context ->

```
retriever =  
vectorstore.as_retriever(search_kwargs={"k": 3})
```

Make generation smarter ->

```
model = ChatOpenAI(model="gpt-4.1-mini",  
temperature=0)
```



Try (lyrics RAG)

Reduce overlap ->

```
CHUNK_OVERLAP = 0
```

Add summaries ->

```
RETRIEVAL_SOURCE = "both"
```

Python

```
RETRIEVAL_SOURCE = "lyrics"
```

```
RETRIEVAL_SOURCE = "summaries"
```

```
RETRIEVAL_SOURCE = "both"
```

Or run from terminal:

bash

```
.venv/bin/python lyrics_rag.py --retrieval-source lyrics
```

```
.venv/bin/python lyrics_rag.py --retrieval-source summaries
```

```
.venv/bin/python lyrics_rag.py --retrieval-source both
```

How can you make the search even better?

Retrieval

lyrics_rag.py --compare --topic "friendship"

Top 3 Retrieval Results

Friendship

1

Black Friday
(pretty like the sun)

cosine similarity 0.36

source: lyrics

2

Ára bátur

cosine similarity 0.32

source: summary

3

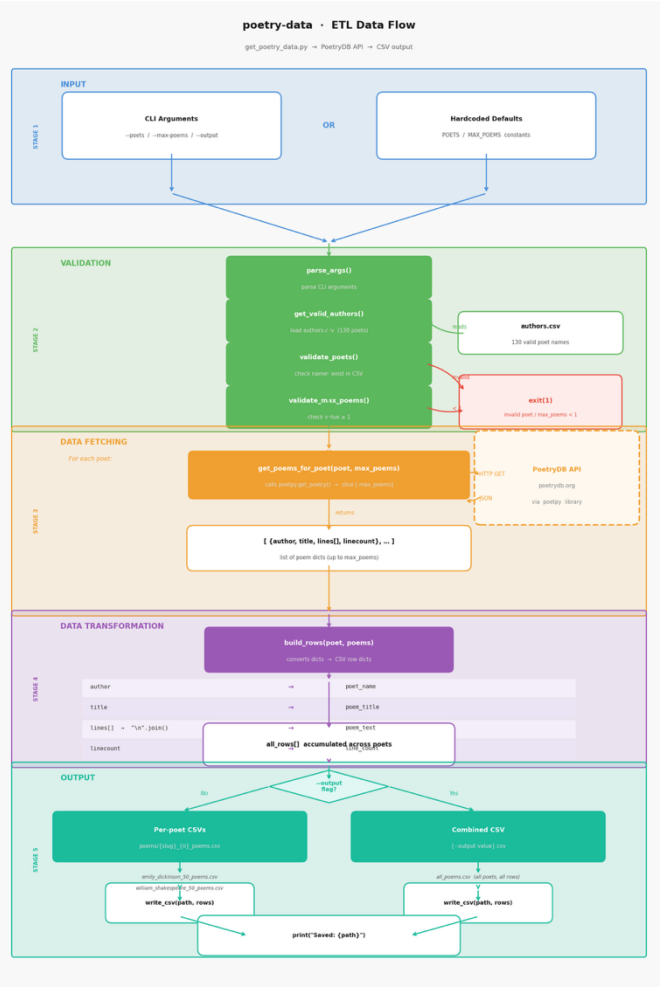
CANCELLED!

cosine similarity 0.29

source: summary

Visual Project Mapping

Claude Code

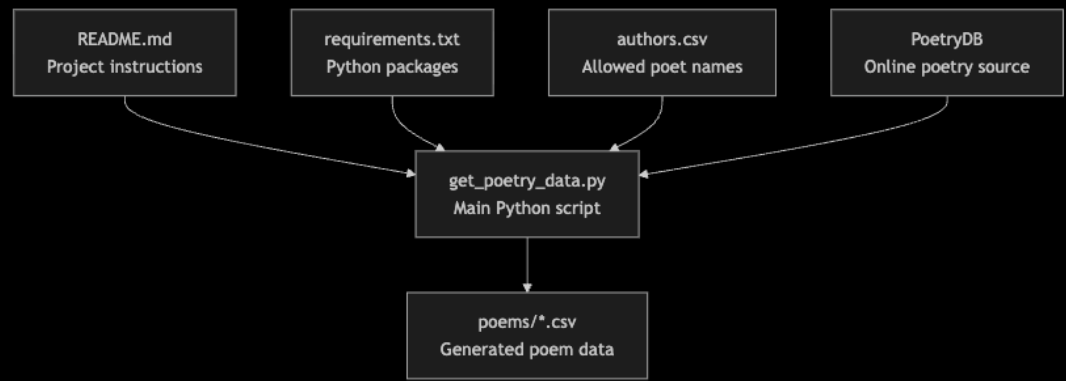


Codex

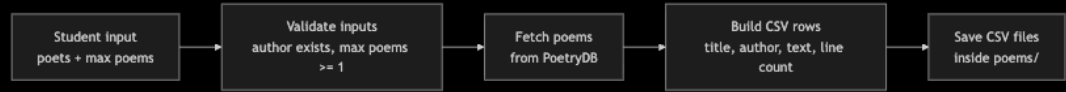
Code Map

This visual map shows how the files and functions in this project fit together.

File Map

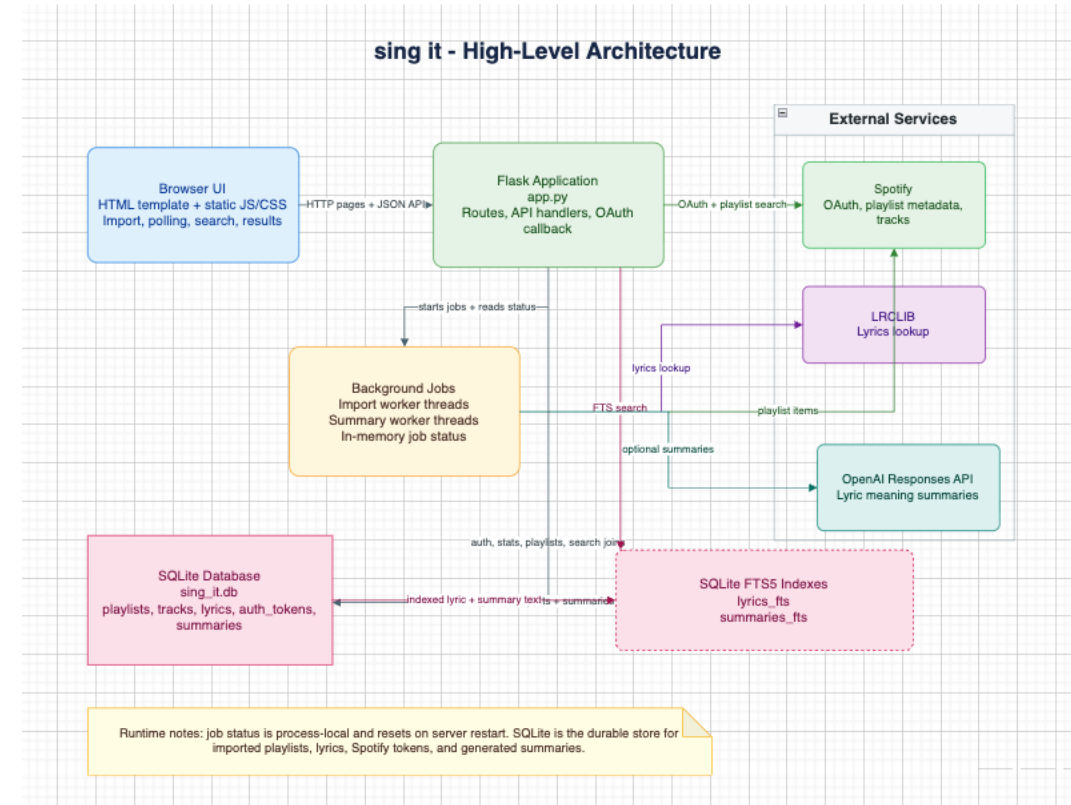
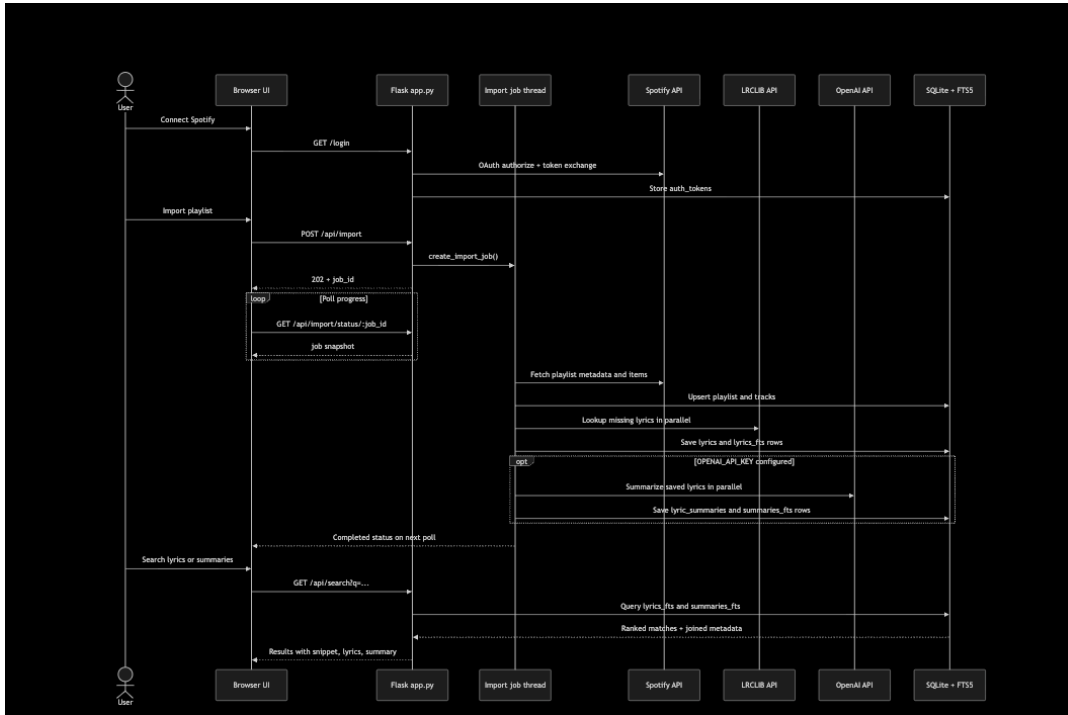


Data Flow



Generate a Code map using Mermaid

Architecture Diagram



Instead of asking for a generic architecture diagram, use the draw.io extension to visualise the project's high-level architecture, request flows, and data model.

Product activity

How to use

Product discovery	Turn rough product ideas into user stories, assumptions, edge cases, risks, and testable hypotheses.
Planning	Break a feature into milestones, implementation steps, dependencies, and acceptance criteria.
Technical exploration	Inspect an unfamiliar codebase and explain its structure, architecture patterns, data models, and authentication flow.
Quick prototyping	Generate a first working version of a feature, app screen, API endpoint, data model, or workflow.
Architecture design	Propose system architecture, module boundaries, data flow, integration points, and trade-offs.
System diagrams	Generate Mermaid diagrams, sequence diagrams, data-flow diagrams, component diagrams, and deployment diagrams from either requirements or existing code.
Backend implementation	Create APIs, database schemas, service logic, authentication flows, background jobs, and integrations.
Refactoring	Clean up duplicated logic, improve naming, split large files, simplify modules, and modernise old code. Claude Code is designed for coordinated multi-file edits.
Testing	Write unit tests, integration tests, regression tests, mocked test data, and test plans. Its official workflow docs include testing, bug fixing, and PR preparation as common use cases.
Code review	Review diffs for correctness, maintainability, security issues, performance problems, and missing tests.
Documentation	Generate README files, API docs, onboarding notes, architecture decision records, release notes, and developer handover material.
Developer onboarding	Explain how a repository works, where to make changes, how to run it locally, and what conventions the project follows.
Git workflows	Create branches, prepare commits, summarize changes, open pull requests, and respond to review comments. Claude Code can work with GitHub/GitLab and command-line tools.
Release preparation	Check what changed, identify risks, update changelogs, suggest smoke tests, and prepare deployment notes.
Maintenance	Upgrade dependencies, remove dead code, improve error handling, add monitoring hooks, and fix recurring issues.
Product analytics support	Add event tracking, define analytics events, inspect instrumentation gaps, and create simple data pipelines or dashboards.
Security review	Look for unsafe defaults, exposed secrets, weak validation, missing error handling, and fragile assumptions.
Experimentation	Build A/B-test variants, feature flags, small MVPs, and disposable prototypes to test product direction quickly.